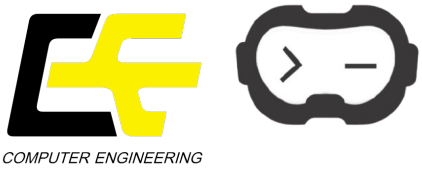


Laboratorium
Robotika dan Sistem Cerdas
Departemen Teknik Komputer
Institut Teknologi Sepuluh Nopember

Laporan Akhir
Praktikum Sistem Mikroprosesor dan Sistem Tertanam

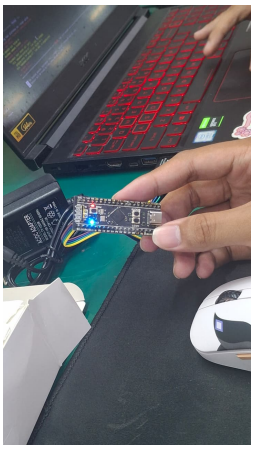
Pengantar Mikrokontroler

Sarah Shafira Maulida - 5024241035

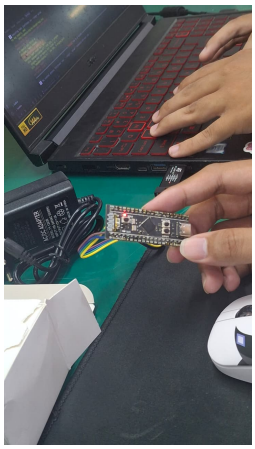
2026

1 Langkah-Langkah Percobaan

1. Menyiapkan alat dan bahan berupa board STM32 Blackpill, ESP32 DevKit, ST-Link, pushbutton, resistor, LDR, breadboard, kabel jumper, dan laptop.
2. Membuat satu project PlatformIO pada Visual Studio Code untuk seluruh percobaan.
3. Melakukan percobaan Blink pada STM32 Blackpill menggunakan framework Arduino dengan menghubungkan board ke komputer melalui ST-Link.

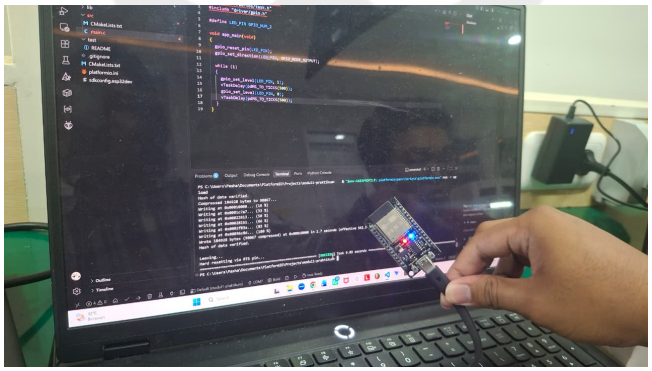


LED ON

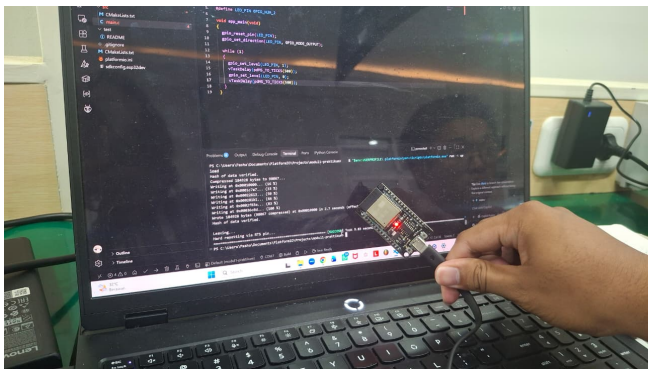


LED OFF

4. Melakukan percobaan Blink pada ESP32 menggunakan framework ESP-IDF dengan mengunggah program melalui kabel USB.

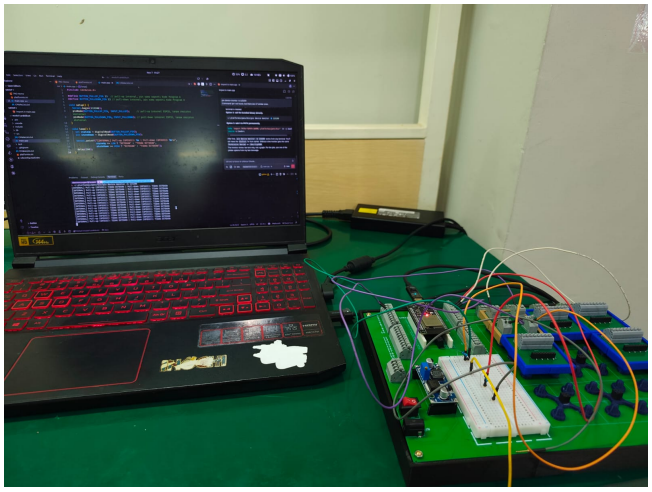


LED ON

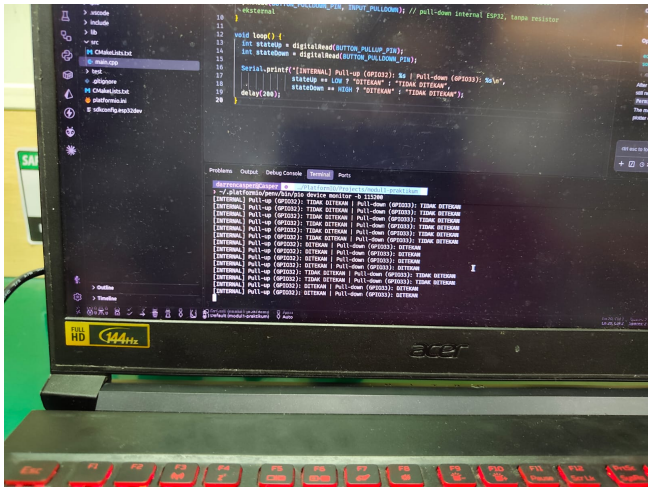


LED OFF

5. Merangkai dua pushbutton pada ESP32 untuk menguji konfigurasi pull-up dan pull-down eksternal, kemudian membandingkannya dengan konfigurasi internal melalui Serial Monitor.

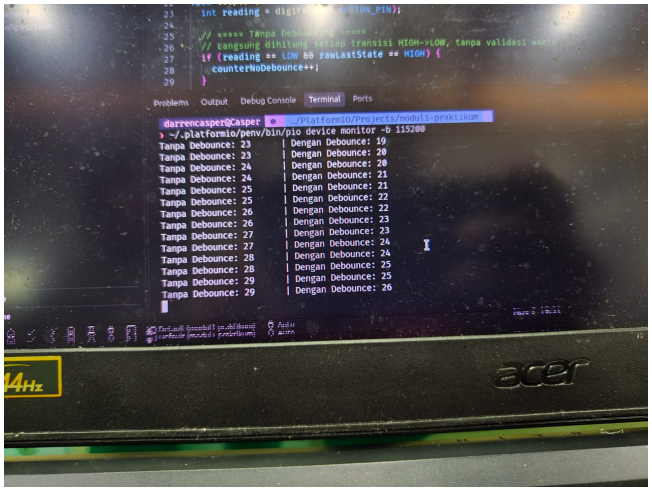


Gambar 1: Rangkaian

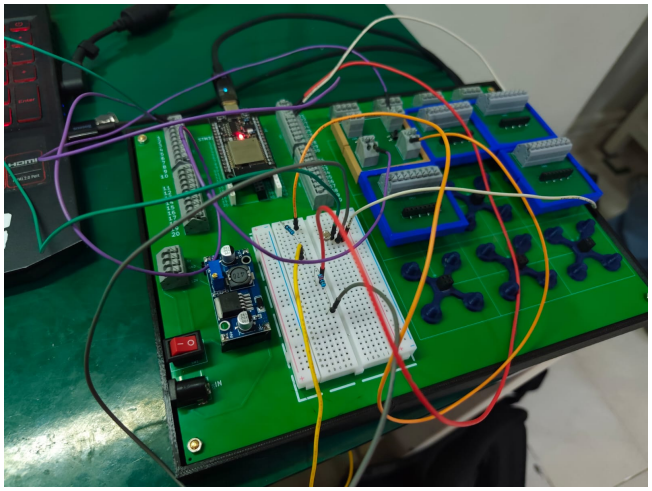


Gambar 2: Serial Monitor

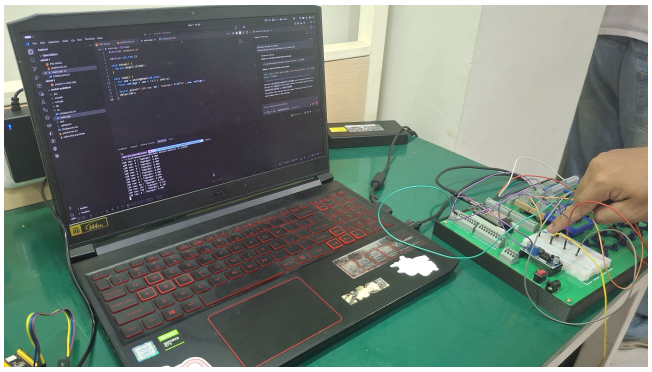
6. Merangkai pushbutton pada GPIO 25 untuk menguji fenomena bouncing dengan membandingkan penghitung tombol sebelum dan sesudah penerapan debouncing menggunakan variasi waktu 10 ms, 50 ms, dan 200 ms.



7. Merangkai LDR dengan resistor 10 kΩ sebagai pembagi tegangan dan menghubungkannya ke GPIO 34 untuk membaca nilai ADC serta tegangan melalui Serial Monitor.



Gambar 3: Rangkaian



Gambar 4: Serial Monitor

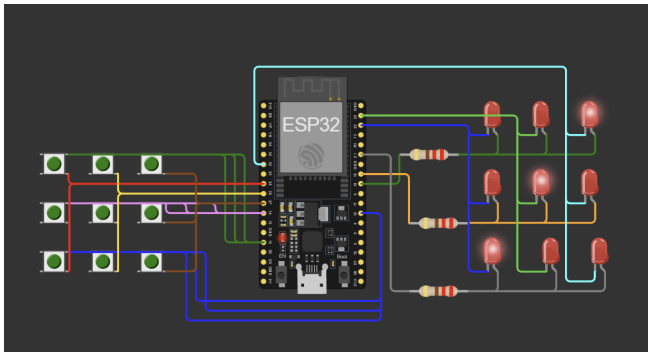
8. Mengamati dan mencatat hasil setiap percobaan, kemudian melakukan analisis terhadap hasil yang diperoleh.

2 Analisis Hasil Percobaan

Berdasarkan percobaan yang telah dilakukan, penggunaan board STM32 Blackpill dan ESP32 berhasil menunjukkan fungsi mikrokontroler dalam mengendalikan GPIO sebagai digital output melalui program Blink. Pada percobaan pertama, LED onboard STM32 berkedip menggunakan framework Arduino, sedangkan pada percobaan kedua ESP32 menjalankan program serupa menggunakan framework ESP-IDF dengan struktur program yang berbeda. Selanjutnya, percobaan pull-up dan pull-down menunjukkan bahwa kondisi logika tombol dipengaruhi oleh konfigurasi resistor, yaitu HIGH saat tidak ditekan pada pull-up dan LOW pada pull-down. Pada percobaan debouncing, pembacaan tombol tanpa penyaringan dapat menghasilkan beberapa kali deteksi akibat bouncing, sedangkan penerapan debouncing membantu menstabilkan pembacaan input dengan mempertimbangkan interval waktu tertentu. Terakhir, percobaan pembacaan ADC menggunakan LDR menunjukkan bahwa perubahan intensitas cahaya dapat menghasilkan perubahan nilai analog yang dibaca melalui GPIO 34 dan dikonversi menjadi tegangan. Secara keseluruhan, percobaan ini menunjukkan penerapan GPIO

sebagai digital output, digital input, dan analog input pada mikrokontroler serta pentingnya konfigurasi perangkat keras dan pemrograman yang tepat untuk memperoleh hasil pembacaan yang sesuai dengan teori.

3 Hasil Tugas Modul



Gambar 5: Rangkaian

```
const int buttonRows[3] = {13, 14, 16};
const int buttonCols[3] = {25, 26, 27};

const int ledRows[3] = {18, 19, 21};
const int ledCols[3] = {22, 23, 32};

const int DEBOUNCE_DELAY = 50;

bool ledState[3][3] = {false};

bool lastReading[3][3];
bool stableState[3][3];

unsigned long lastDebounceTime[3][3] = {0};

void setup() {
  Serial.begin(115200);
  for (int i = 0; i < 3; i++) {
    pinMode(buttonRows[i], OUTPUT);
    digitalWrite(buttonRows[i], HIGH);
  }

  for (int j = 0; j < 3; j++) {
    pinMode(buttonCols[j], INPUT_PULLUP);
  }

  for (int i = 0; i < 3; i++) {
    pinMode(ledRows[i], OUTPUT);
    digitalWrite(ledRows[i], LOW);
  }
```

```
31
32    for (int j = 0; j < 3; j++) {
33      pinMode(ledCols[j], OUTPUT);
34      digitalWrite(ledCols[j], HIGH);
35    }
36
37    for (int i = 0; i < 3; i++) {
38      for (int j = 0; j < 3; j++) {
39        lastReading[i][j] = HIGH;
40        stableState[i][j] = HIGH;
41      }
42    }
43  }
44
45  void readButtons() {
46    for (int i = 0; i < 3; i++) {
47      digitalWrite(buttonRows[i], LOW);
48      delayMicroseconds(50);
49
50      for (int j = 0; j < 3; j++) {
51        bool reading = digitalRead(buttonCols[j]);
52
53        if (reading != lastReading[i][j]) {
54          lastDebounceTime[i][j] = millis();
55        }
56
57        if (millis() - lastDebounceTime[i][j] > DEBOUNCE_DELAY) {
58          if (reading != stableState[i][j]) {
59            stableState[i][j] = reading;
60
61            if (stableState[i][j] == LOW) {
62              ledState[i][j] = !ledState[i][j];
63              Serial.printf("Tombol baris %d kolom %d ditekan\n", i + 1, j +
    1);
64            }
65          }
66        }
67        lastReading[i][j] = reading;
68      }
69      digitalWrite(buttonRows[i], HIGH);
70    }
71  }
72
73  void displayLED() {
74    for (int i = 0; i < 3; i++) {
75      digitalWrite(ledRows[i], LOW);
76
77      for (int j = 0; j < 3; j++) {
78        digitalWrite(ledCols[j], HIGH);
79      }
```

```
80
81      for (int j = 0; j < 3; j++) {
82        if (ledState[i][j]) {
83          digitalWrite(ledCols[j], LOW);
84        }
85      }
86
87      digitalWrite(ledRows[i], HIGH);
88      delay(2);
89      digitalWrite(ledRows[i], LOW);
90    }
91  }
92
93  void loop() {
94    readButtons();
95    displayLED();
96  }
```

1. Debouncing tetap diperlukan pada rangkaian matrix pushbutton 3x3 karena setiap tombol memiliki kontak mekanik yang dapat mengalami getaran saat bersentuhan atau terlepas. Getaran tersebut menyebabkan munculnya beberapa perubahan sinyal HIGH dan LOW dalam waktu singkat (*bouncing*), sehingga satu kali tekanan dapat terbaca sebagai beberapa input oleh mikrokontroler. Meskipun umumnya hanya satu tombol yang ditekan dalam satu waktu, kondisi tersebut tetap dapat menyebabkan perubahan status LED yang tidak sesuai. Oleh karena itu, debouncing diterapkan pada setiap tombol dengan memastikan sinyal berada dalam kondisi stabil selama 50 ms sebelum perubahan status LED diproses.

2. Konfigurasi yang digunakan pada rangkaian matrix pushbutton adalah pull-up internal dengan menggunakan fungsi `INPUT_PULLUP` pada pin kolom ESP32. Dalam konfigurasi ini, pin GPIO akan berada pada kondisi HIGH ketika tombol tidak ditekan karena terhubung secara internal ke tegangan 3.3V. Sebaliknya, ketika tombol ditekan, pin akan terhubung ke GND melalui baris yang sedang diaktifkan sehingga terbaca LOW. Konfigurasi ini diperlukan untuk menghindari kondisi *floating*, yaitu keadaan ketika pin GPIO tidak memiliki level tegangan yang pasti sehingga dapat menghasilkan pembacaan HIGH atau LOW secara tidak menentu. Kondisi tersebut dapat menyebabkan kesalahan pembacaan tombol dan perubahan status LED yang tidak diinginkan.

3. Digital input dan analog input memiliki perbedaan pada jenis sinyal yang dibaca. Digital input hanya mengenali dua kondisi logika, yaitu HIGH dan LOW, sehingga sesuai digunakan untuk membaca status pushbutton yang hanya memiliki kondisi ditekan atau tidak ditekan. Sementara itu, analog input menggunakan ADC (*Analog-to-Digital Converter*) untuk mengubah tegangan analog menjadi nilai digital yang dapat diproses oleh mikrokontroler. Pada ESP32 dengan resolusi ADC 12-bit, hasil pembacaan memiliki rentang nilai 0–4095 sehingga mampu merepresentasikan berbagai tingkat tegangan. Apabila salah satu tombol digantikan dengan LDR, perubahan intensitas cahaya akan memengaruhi resistansi dan tegangan pada rangkaian pembagi tegangan, sehingga menghasilkan perubahan nilai ADC. Dengan demikian, ADC dapat merepresentasikan lebih dari dua kondisi karena mampu membaca variasi tegangan secara

bertingkat, bukan hanya HIGH dan LOW.

4 Kesimpulan

Praktikum ini memberikan pemahaman mengenai penggunaan GPIO pada STM32 dan ESP32 sebagai digital output, digital input, dan analog input. Selain itu, praktikan memahami konfigurasi pull-up dan pull-down, penerapan debouncing untuk menstabilkan pembacaan tombol, serta penggunaan ADC untuk membaca sensor LDR. Hasil percobaan secara umum sesuai dengan teori yang telah dipelajari.